

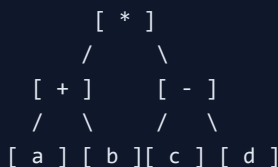
Applications of Binary Tree: Expression Tree & Huffman Encoding — Data Structures

Definition: Binary Trees are applied extensively in compilers (Expression Trees for arithmetic parsing) and data compression algorithms (**Huffman Encoding** for prefix-free lossless entropy coding).

1. Application 1: Expression Trees

An **Expression Tree** is a binary tree where **internal nodes are operators** (+, -, *, /) and **leaf nodes are operands** (a, b, 5, 10).

Expression: (a + b) * (c - d)



Traversals of Expression Tree:

1. **Inorder Traversal:** Yields **Infix Expression** with parentheses -> ((a + b) * (c - d))
2. **Preorder Traversal:** Yields **Prefix (Polish) Expression** -> * + a b - c d
3. **Postorder Traversal:** Yields **Postfix (Reverse Polish) Expression** -> a b + c d - *

Constructing Expression Tree from Postfix Expression:

1. Read postfix string token by token.
2. If token is an **operand**: Create node and push to Stack.
3. If token is an **operator**: Pop two nodes **T1** and **T2** from stack, create a new operator node with **left = T2** and **right = T1**, then push the operator node back onto Stack.

2. Application 2: Huffman Encoding (Data Compression)

Huffman Coding is a **greedy algorithm** that assigns variable-length binary codes to characters based on their frequency of occurrence. More frequent characters get shorter codes.

Step-by-Step Huffman Tree Construction:

1. Count frequency of each unique character in input text.
2. Insert all characters into a **Min-Priority Queue (Min-Heap)**.
3. While Priority Queue has more than 1 node:
 - Extract the two nodes with the smallest frequencies: **Left = extractMin()**, **Right = extractMin()**.

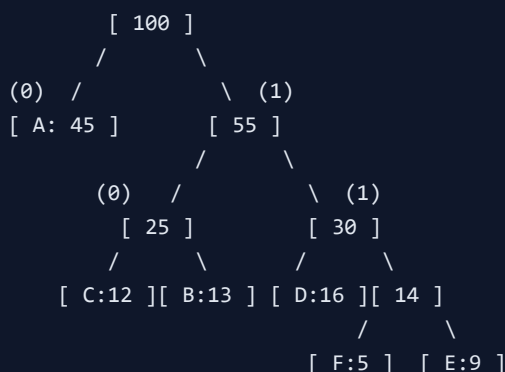
- Create a new internal parent node with frequency = `Left.freq + Right.freq`.
- Re-insert parent node into Min-Priority Queue.

4. The remaining single node is the **Root of Huffman Tree**.

5. Assign binary `0` to all left branches and binary `1` to all right branches.

Sample Character Frequencies:

A: 45, B: 13, C: 12, D: 16, E: 9, F: 5



Resulting Prefix-Free Codes:

- **A** -> `0` (1 bit)
- **C** -> `100` (3 bits)
- **B** -> `101` (3 bits)
- **D** -> `110` (3 bits)
- **F** -> `1110` (4 bits)
- **E** -> `1111` (4 bits)

3. Must-Write Points for Exams

- Huffman codes are **Prefix-Free Codes** (no character code is a prefix of any other code), enabling unambiguous decompression.
- In Expression Trees, leaves are operands and non-leaves are operators.
- Huffman Coding achieves optimal minimum weighted path length (lossless compression).

4. Quick Recall Flow

Expression Tree: Leaves=Operands, Internal=Operators | Huffman Coding: Min-Heap -> Merge 2 Lowest Frequencies -> 0 Left / 1 Right -> Variable Length Prefix Code